Lab 12: File System and Implementation

12.1: Symbolic and hard Link

Linux

`ln --help`

```
Make links between files.

Usage: ln [OPTION]... [-T] TARGET LINK_NAME
       ln [OPTION]... TARGET
       ln [OPTION]... TARGET... DIRECTORY
       ln [OPTION]... -t DIRECTORY TARGET...

Arguments:
  <files>...

Options:
      --backup[=<CONTROL>]           make a backup of each existing destination file
  -b                                 like --backup but does not accept an argument
  -f, --force                        remove existing destination files
  -i, --interactive                  prompt whether to remove existing destination files
  -n, --no-dereference               treat LINK_NAME as a normal file if it is a
                                     symbolic link to a directory
  -L, --logical                      follow TARGETs that are symbolic links
  -P, --physical                     make hard links directly to symbolic links
  -s, --symbolic                     make symbolic links instead of hard links
  -S, --suffix <SUFFIX>              override the usual backup suffix
  -t, --target-directory <DIRECTORY>  specify the DIRECTORY in which to create the links
  -T, --no-target-directory          treat LINK_NAME as a normal file always
  -r, --relative                     create symbolic links relative to link location
  -v, --verbose                      print name of each linked file
  -h, --help                         Print help
  -V, --version                      Print version

In the 1st form, create a link to TARGET with the name LINK_NAME.
In the 2nd form, create a link to TARGET in the current directory.
In the 3rd and 4th forms, create links to each TARGET in DIRECTORY.
```

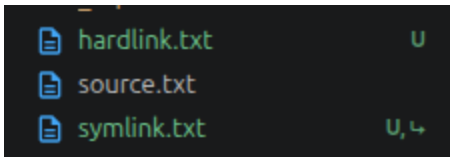
```
Create hard links by default, symbolic links with --symbolic.
By default, each destination (name of new link) should not already exist.
When creating hard links, each TARGET must exist. Symbolic links
can hold arbitrary text; if later resolved, a relative link is
interpreted in relation to its parent directory.

The backup suffix is '~', unless set with --suffix or SIMPLE_BACKUP_SUFFIX.
The version control method may be selected via the --backup option or through
the VERSION_CONTROL environment variable.  Here are the values:

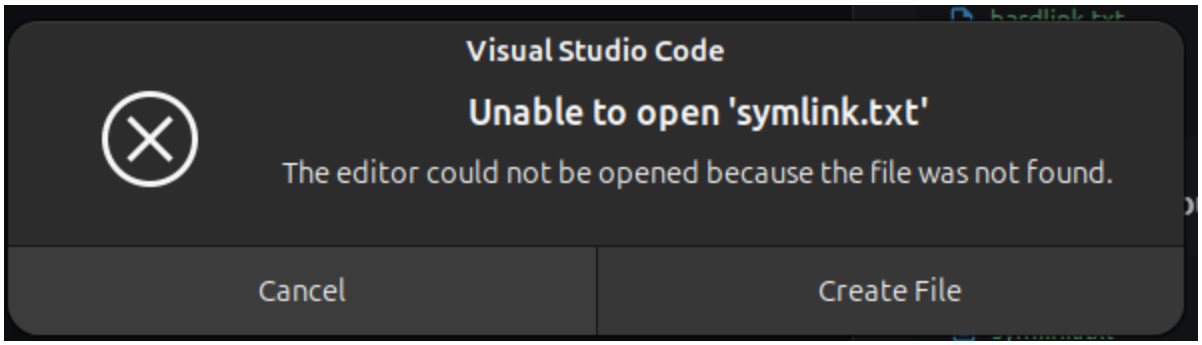
  none, off       never make backups (even if --backup is given)
  numbered, t     make numbered backups
  existing, nil   numbered if numbered backups exist, simple otherwise
  simple, never   always make simple backups
```

Create hard/soft links

- soft: `ln -s source.txt symlink.txt`
- hard: `ln source.txt hardlink.txt`



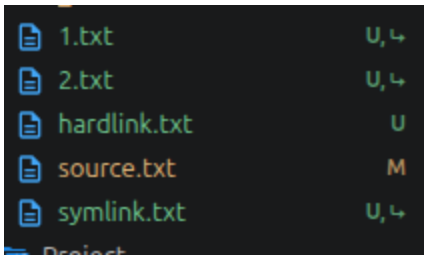
After delete the source file



- hardlink.txt still works.
- symlink.txt cannot access its deleted target and are now unaccesable.

Create level 1 and create level 2 link

- level1: `ln -s source.txt 1.txt`
- level2: `ln -s 1.txt 2.txt`



After delete the source file

Both level 1 and 2 link are now unaccesable.

After delete level 1 link

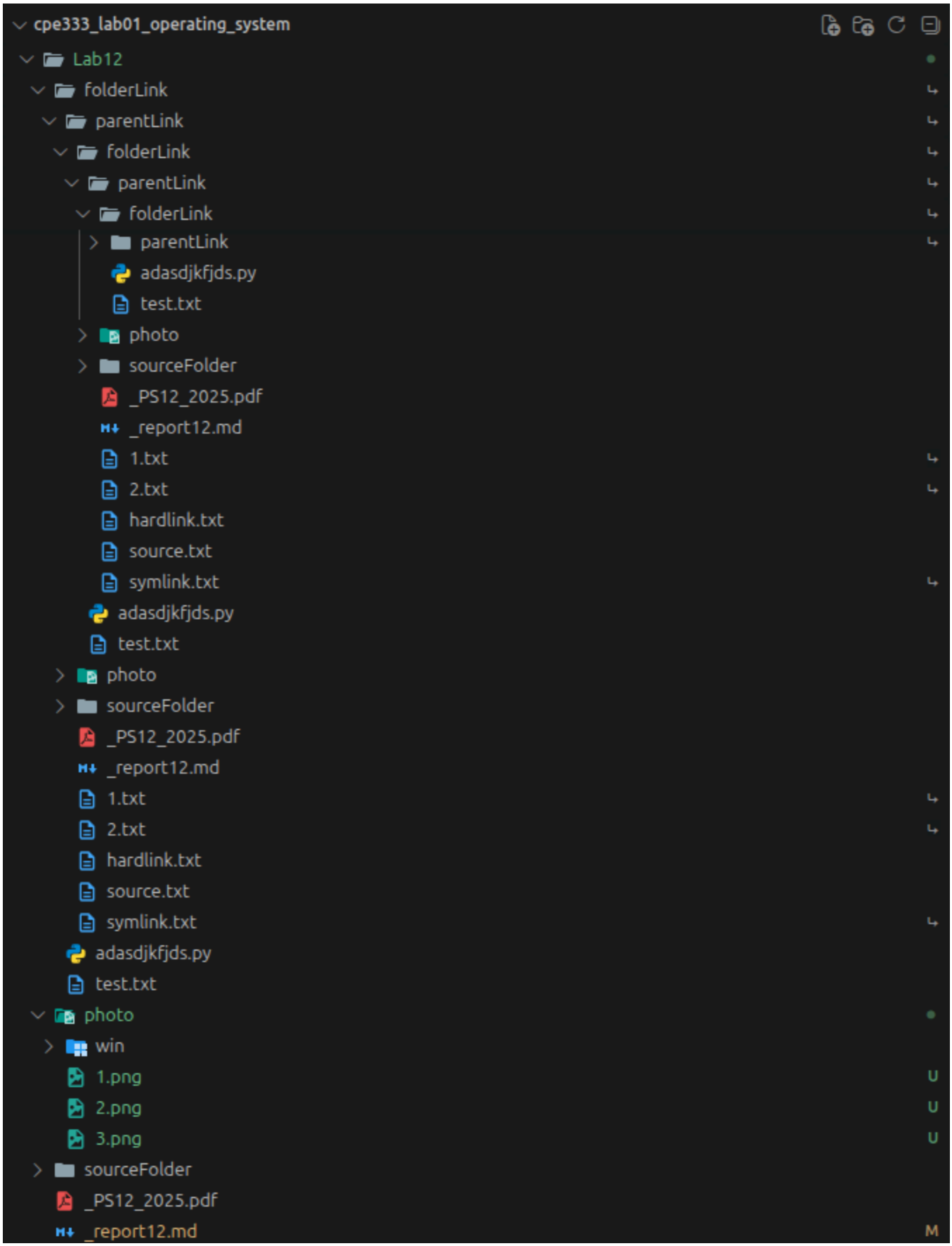
Level 2 link is now unaccesable.

Create a link to directory or folder.

```
ln -s folderA folderLink
```

Create a link that create a loop back to upper-level folder.

```
ln -s .. folderA/parentLink
```


cpe333_lab01_operating_system
Lab12
folderLink
parentLink
folderLink
parentLink
folderLink
parentLink
adasdjkfjds.py
test.txt
photo
sourceFolder
_PS12_2025.pdf
_report12.md
1.txt
2.txt
hardlink.txt
source.txt
symlink.txt
adasdjkfjds.py
test.txt
photo
sourceFolder
_PS12_2025.pdf
_report12.md
1.txt
2.txt
hardlink.txt
source.txt
symlink.txt
adasdjkfjds.py
test.txt
photo
win
1.png
2.png
3.png
sourceFolder
_PS12_2025.pdf
_report12.md

```
ls -R
```

```
.:
1.txt  2.txt  _PS12_2025.pdf  _report12.md  folderLink  hardlink.txt  photo  source.txt  sourceI

./photo:
1.png  2.png  3.png  4.png  win

./photo/win:
winDelSource1.png  winFolder.png  winLoop.png  winLv.png  windowLink1.png

./sourceFolder:
adasdjkfjds.py  parentLink  test.txt
```

Window

mklink

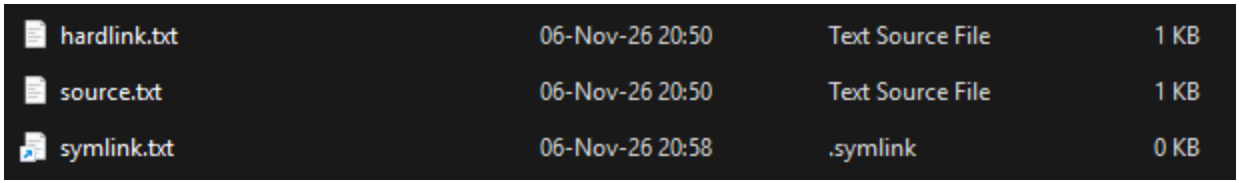
```
Creates a symbolic link.

MKLINK [[/D] | [/H] | [/J]] Link Target

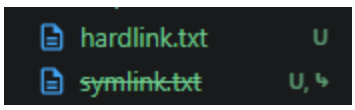
        /D      Creates a directory symbolic link.  Default is a file
                symbolic link.
        /H      Creates a hard link instead of a symbolic link.
        /J      Creates a Directory Junction.
        Link    Specifies the new symbolic link name.
        Target  Specifies the path (relative or absolute) that the new link
                refers to.
```

Create hard/soft links

- soft: `mklink symlink.txt source.txt`
- hard: `mklink /H hardlink.txt source.txt`



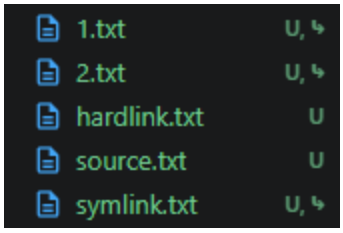
After delete the source file



- hardlink.txt still works.
- symlink.txt cannot access its deleted target and are now unaccesable.

Create level 1 and create level 2 link

- level1: `mklink 1.txt source.txt`
- level2: `mklink 2.txt 1.txt`



After delete the source file

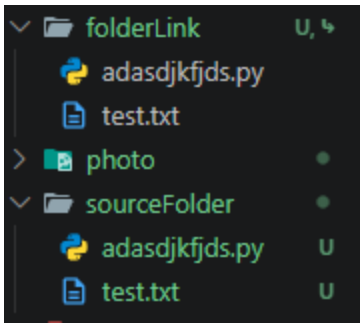
Both level 1 and 2 link are now unaccesable.

After delete level 1 link

Level 2 link is now unaccesable.

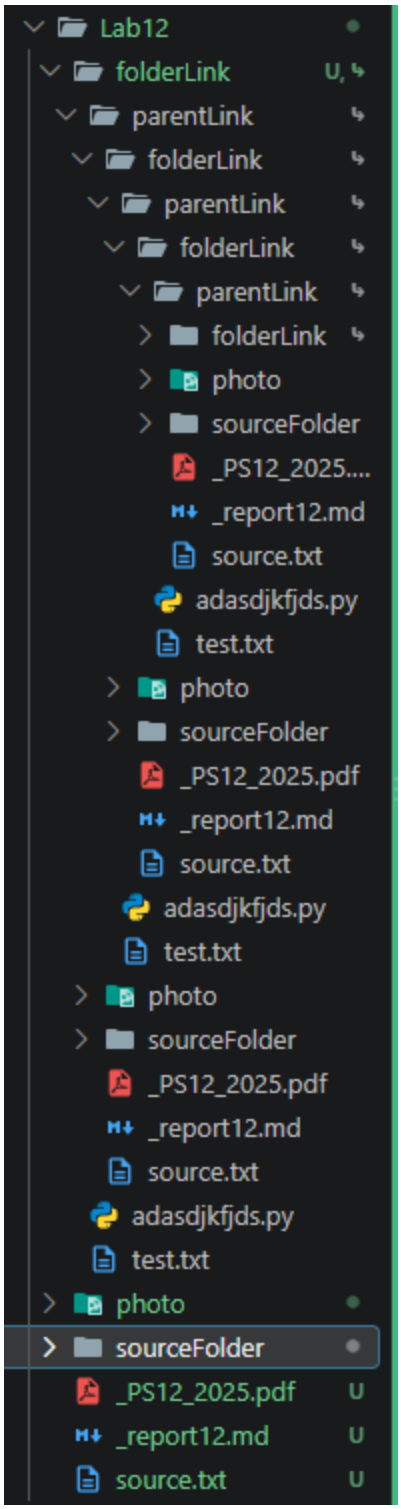
Create a link to directory or folder.

```
mklink /D folderLink sourceFolder
```



Create a link that create a loop back to upper-level folder.

```
mklink /D sourceFolder\parentLink ..
```


Lab12
folderLink U,
parentLink
folderLink
parentLink
folderLink
parentLink
folderLink
photo
sourceFolder
_PS12_2025....
_report12.md
source.txt
adasdjkfjds.py
test.txt
photo
sourceFolder
_PS12_2025.pdf
_report12.md
source.txt
adasdjkfjds.py
test.txt
photo
sourceFolder
_PS12_2025.pdf
_report12.md
source.txt
adasdjkfjds.py
test.txt
photo
sourceFolder
_PS12_2025.pdf U
_report12.md U
source.txt U

```
dir /s
```

```
 Directory of D:\KMUTT\Year 3\Operating System\Lab12\folderLink\parentLink\folderLink\parentLink

06-Nov-26  21:33    <DIR>          .
06-Nov-26  21:34    <DIR>          ..
06-Nov-26  21:31                 0 adasdjkfjds.py
06-Nov-26  21:33    <SYMLINKD>     parentLink [..]
06-Nov-26  21:29                33 test.txt
               2 File(s)             33 bytes

 Directory of D:\KMUTT\Year 3\Operating System\Lab12\folderLink\parentLink\folderLink\parentLink

06-Nov-26  21:34    <DIR>          .
06-Nov-26  21:17    <DIR>          ..
06-Nov-26  21:30    <SYMLINKD>     folderLink [sourceFolder]
06-Nov-26  21:35    <DIR>          photo
06-Nov-26  21:27                 3 source.txt
06-Nov-26  21:33    <DIR>          sourceFolder
06-Nov-26  20:07           135,428 _PS12_2025.pdf
06-Nov-26  21:35             1,469 _report12.md
               3 File(s)        136,900 bytes

 Directory of D:\KMUTT\Year 3\Operating System\Lab12\folderLink\parentLink\folderLink\parentLink

06-Nov-26  21:33    <DIR>          .
06-Nov-26  21:34    <DIR>          ..
06-Nov-26  21:31                 0 adasdjkfjds.py
06-Nov-26  21:33    <SYMLINKD>     parentLink [..]
06-Nov-26  21:29                33 test.txt
               2 File(s)             33 bytes

 Directory of D:\KMUTT\Year 3\Operating System\Lab12\folderLink\parentLink\folderLink\parentLink

06-Nov-26  21:35    <DIR>          .
06-Nov-26  21:34    <DIR>          ..
06-Nov-26  21:07             2,229 winDelSource1.png
06-Nov-26  21:01             6,890 windowLink1.png
06-Nov-26  21:31             7,557 winFolder.png
06-Nov-26  21:35            29,419 winLoop.png
06-Nov-26  21:15             3,572 winLv.png
               5 File(s)         49,667 bytes
```

it goes on for a while...

Discussion

1. Hard links and symbolic links to files

Create two different kinds of links. A hard link is another directory entry for the same file data. Removing the original name does not remove the data while another hard-link name remains, so `hardlink.txt` can still be opened. A symbolic link stores a path to its target instead of sharing the target's file data. When `source.txt` is removed, `symlink.txt` remains but its stored path no longer resolves, so it is a dangling link. Windows show the same practical distinction on the tested filesystem.

2. Links made through another link

In both experiments, `2.txt` points to `1.txt` , and `1.txt` points to `source.txt` . Accessing `2.txt` therefore requires every link in that path to resolve. Deleting `source.txt` makes both links unusable even though the link files remain. Deleting `1.txt` also makes `2.txt` dangling because its immediate target is gone. Relative symbolic-link targets are resolved from the directory containing each link. This is why the relative names in these examples work while all three files are in the same directory.

3. Directory links and recursive listing

The Linux `ln -s` command and Windows `mklink /D` can create links to directories. The `parentLink` link points to `..` , while `folderLink` points to the source directory, creating a path that can return to the same directories repeatedly. In the recorded Linux output, `ls -R` lists the link itself but does not recursively follow the linked directory by default. Windows `dir /s` in this experiment follows the directory links and prints the same folders under increasingly long paths. Recursive traversal through a loop can produce excessive output or run into path and system limits. remove one of the directory links to break the loop before doing further recursive listings. A directory symbolic link is not a hard link, ordinary hard links to directories are generally prohibited because they could create such cycles.

4. Creating a link across drives or filesystems

A symbolic link can refer to a target on another drive or filesystem, provided the target path is valid and the operating system and filesystem support symbolic links. For example, from Windows Command Prompt, a link located on `D:` can refer to a source on `C:` . Deleting or moving the source leaves a link whose target cannot be found.

A hard link cannot cross filesystem/volume boundaries. `ln` fails when source and link are on different mounted filesystems, and Windows `mklink /H` requires both names to be on the same volume. Thus, a link on `D:` to a source on `C:` can be made as a symbolic link, but not as a hard link.

12.2: Linux's File Type: Named Pipe

Create a named pipe (FIFO) named `pipe` :

```
mkfifo pipe
ls -la
ls -la pipe
```

```
ls -la
```

```
total 176
drwxrwxr-x  4 pai pai   4096 Nov  7 10:14 .
drwxrwxr-x 13 pai pai   4096 Nov  7 10:14 ..
lrwxrwxrwx  1 pai pai     10 Nov  6 22:18 1.txt -> source.txt
lrwxrwxrwx  1 pai pai      5 Nov  6 22:19 2.txt -> 1.txt
-rw-rw-r--  1 pai pai 135428 Nov  6 22:02 _PS12_2025.pdf
-rw-rw-r--  1 pai pai  12294 Nov  7 10:15 _report12.md
lrwxrwxrwx  1 pai pai     53 Nov  7 09:47 diskHardlink.txt -> C:/Users/paipa/Downloads/sprint-1·
lrwxrwxrwx  1 pai pai     12 Nov  6 22:02 folderLink -> sourceFolder
-rw-rw-r--  2 pai pai      5 Nov  6 22:14 hardlink.txt
drwxrwxr-x  3 pai pai   4096 Nov  6 22:21 photo
prw-rw-r--  1 pai pai      0 Nov  7 10:14 pipe
-rw-rw-r--  1 pai pai     13 Nov  6 22:15 source.txt
drwxrwxr-x  2 pai pai   4096 Nov  7 10:04 sourceFolder
lrwxrwxrwx  1 pai pai     10 Nov  6 22:12 symlink.txt -> source.txt
```

```
prw-rw-r--  1 pai pai      0 Nov  7 10:14 pipe
```

Cat Pipe Experiment

Open two terminals and run two command.

```
cat > pipe
```

```
cat pipe
```

Type `hello linux in read` from fill `cat pipe` and `hello linux from write` in `cat > pip`



Explanation

A named pipe, also called a FIFO (first in, first out), is a special filesystem object that lets separate processes communicate. Unlike a regular file, it does not store the transferred data for later reading. The kernel passes bytes written to the pipe to a process reading from it, in order, using a temporary buffer.

In the `ls -la` output, the first character in `prw-rw-r--` is `p`, identifying `pipe` as a named pipe. Its displayed size is `0` because it is not a regular file containing stored bytes. Data is consumed through the pipe instead.

`cat > pipe` copies standard input into the pipe, while `cat pipe` copies data from the pipe to its standard output. Opening a FIFO for reading or writing normally waits until a process opens the other end, which is why the two terminals must run their commands together. The reader receives each line as the writer sends it. Ctrl+D ends the writer's terminal input, causing the writing `cat` to close the FIFO; once the buffered data has been read, the reader gets end-of-file and exits.

12.3: Reading and Writing Files

```
echo hello > foo

strace cat foo


 splice(3, NULL, 5, NULL, 1048576, 0)    = 6
 splice(4, NULL, 1, NULL, 6, 0hello
 )          = 6
 splice(3, NULL, 5, NULL, 1048576, 0)    = 0
 close(5)                                = 0
 close(4)                                = 0
 close(3)                                = 0
```

Explanation

My output uses `splice` instead of the `read` and `write` calls shown in the lab example, but it shows the same file-to-terminal transfer. The descriptors in my trace are `3` for the opened file `foo`, `1` for standard output, and `4` and `5` for the temporary pipe's read and write ends.

In my output, the numbers `{3, 4, 6, 1048576}` mean: `3` is the file descriptor for `foo`, `4` is the temporary pipe's read end descriptor. `6` is the number of bytes transferred (`hello` + 1) and `1048576` is the maximum number of bytes requested in a `splice` call. Unlike the handout's `32768`, this transfer limit is much larger than the six bytes in the file, so only six bytes are actually moved.

1. `splice(3, NULL, 5, NULL, 1048576, 0) = 6` moves six bytes from `foo` (descriptor `3`) into the pipe's write end (descriptor `5`). This performs the equivalent reading operation shown by `read(3, "hello\n", 32768) = 6` in the handout. Unlike `read()`, `splice()` transfers data through a pipe without copying it into a userspace buffer. The requested maximum is `1048576` bytes, but only six bytes are available.
2. `splice(4, NULL, 1, NULL, 6, 0) = 6` moves those six bytes from the pipe's read end (descriptor `4`) to standard output (descriptor `1`). This corresponds to `write(1, "hello\n", 6) = 6` in the handout. The six bytes are `hello` followed by the newline added by `echo`.
3. `splice(3, NULL, 5, NULL, 1048576, 0) = 0` attempts to transfer more data from `foo` into the pipe but returns `0`, indicating end-of-file (EOF). This is equivalent to the handout's `read(3, "", 32768) = 0`.
4. `close(5)`, `close(4)` and `close(3)` close the pipe's write end, read end, and the opened file, respectively. The handout only shows `close(3)` because its simpler `read()` / `write()` approach does not require a temporary pipe.

```
execve("/usr/bin/cat", ["cat", "foo"], 0x7ffca232c0a8 /* 82 vars */) = 0
brk(NULL)                               = 0x5c000f255000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x708dbfb76000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=72063, ...}) = 0
mmap(NULL, 72063, PROT_READ, MAP_PRIVATE, 3, 0) = 0x708dbfb64000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x708dbfb31000
mmap(0x708dbfb38000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7000
mmap(0x708dbfb59000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x708
mmap(0x708dbfb60000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2f000
mmap(0x708dbfb62000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=187120, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x708dbfb03000
mmap(0x708dbfb07000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000
mmap(0x708dbfb2b000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x708
mmap(0x708dbfb2f000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2c000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=1198376, ...}) = 0
mmap(NULL, 1200152, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x708dbf9dd000
mmap(0x708dbf9ef000, 655360, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1200
mmap(0x708dbfa8f000, 466944, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xb2000) = 0x708
mmap(0x708dbfb01000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x12300
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2190608, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x708dbf600000
mmap(0x708dbf628000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x280
mmap(0x708dbf7c0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c0000) = 0x70
mmap(0x708dbf80e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20e0
mmap(0x708dbf814000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
```

```
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=699056, ...}) = 0
mmap(NULL, 701112, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x708dbf931000
mmap(0x708dbf934000, 507904, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3000
mmap(0x708dbf9b0000, 176128, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7f000) = 0x708
mmap(0x708dbf9db000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xa9000
close(3)                                = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x708dbf92f000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x708dbf92c000
arch_prctl(ARCH_SET_FS, 0x708dbf92cb00) = 0
set_tid_address(0x708dbf92d128)         = 26591
set_robust_list(0x708dbf92cde0, 24)     = 0
rseq(0x708dbf92c680, 0x21, 0, 0x53053053) = 0
mprotect(0x708dbf80e000, 16384, PROT_READ) = 0
mprotect(0x708dbf9db000, 4096, PROT_READ) = 0
mprotect(0x708dbfb01000, 4096, PROT_READ) = 0
mprotect(0x708dbfb2f000, 4096, PROT_READ) = 0
mprotect(0x708dbfb60000, 4096, PROT_READ) = 0
mprotect(0x5bffe66c1000, 1462272, PROT_READ) = 0
mprotect(0x708dbfbbb000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\x61\x54\x50\x5a\x19\x13\x1a\x98", 8, GRND_NONBLOCK) = 8
munmap(0x708dbfb64000, 72063)           = 0
statfs("/sys/fs/selinux", 0x7ffe75b6ea60) = -1 ENOENT (No such file or directory)
statfs("/selinux", 0x7ffe75b6ea60)      = -1 ENOENT (No such file or directory)
brk(NULL)                               = 0x5c000f255000
brk(0x5c000f276000)                     = 0x5c000f276000
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tpr"..., 1024) = 369
read(3, "", 1024)                       = 0
close(3)                                = 0
access("/etc/selinux/config", F_OK)     = -1 ENOENT (No such file or directory)
fcntl(0, F_GETFD)                       = 0
fcntl(1, F_GETFD)                       = 0
fcntl(2, F_GETFD)                       = 0
rt_sigaction(SIGPIPE, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
poll([{fd=0, events=0}, {fd=1, events=0}, {fd=2, events=0}], 3, 0) = 0 (Timeout)
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_RESTART, sa_r
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
```

```
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "5bffe5d58000-5bffe5e4a000 r--p 0"..., 1024) = 1024
read(3, "0 08:12 6293000                 "..., 1024) = 1024
read(3, "00 08:12 6293003                "..., 1024) = 1024
read(3, "             /usr/lib/x86_64-lin"..., 1024) = 1024
read(3, "2 6292997                     /us"..., 1024) = 650
lseek(3, 0, SEEK_CUR)                   = 4746
lseek(3, 4662, SEEK_SET)                = 4662
close(3)                                = 0
sched_getaffinity(26591, 32, [0 1 2 3 4 5 6 7 8 9 10 11]) = 8
rt_sigaction(SIGSEGV, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
sigaltstack(NULL, {ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=0}) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x708dbfb7
mprotect(0x708dbfb73000, 4096, PROT_NONE) = 0
sigaltstack({ss_sp=0x708dbfb74000, ss_flags=0, ss_size=8192}, NULL) = 0
gettid()                                = 26591
rt_sigaction(SIGSEGV, {sa_handler=0x5bffe607d0c0, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA
rt_sigaction(SIGBUS, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGBUS, {sa_handler=0x5bffe607d0c0, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_
prctl(PR_GET_AUXV, 0x7ffe75b6e550, 0x200, 0, 0) = 448
statx(AT_FDCWD, "/usr/share/coreutils/locales/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=
statx(AT_FDCWD, "/usr/share/coreutils/locales/uucore", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_ma
openat(AT_FDCWD, "/usr/share/coreutils/locales/uucore/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
statx(3, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, s
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32)                         = 0
close(3)                                = 0
statx(AT_FDCWD, "/usr/share/coreutils/locales/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = -1 ENOENT (
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_RESTART, sa_r
rt_sigaction(SIGSEGV, {sa_handler=SIG_DFL, sa_mask=~[RTMIN RT_1], sa_flags=SA_RESTORER, sa_resto
rt_sigaction(SIGBUS, {sa_handler=SIG_DFL, sa_mask=~[RTMIN RT_1], sa_flags=SA_RESTORER, sa_restor
ioctl(1, TCGETS2, {c_iflag=BRKINT|ICRNL|IXON|IXANY|IMAXBEL|IUTF8, c_oflag=NL0|CR0|TAB0|BS0|VT0|F
brk(0x5c000f297000)                     = 0x5c000f297000
statx(AT_FDCWD, "foo", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, stx_a
openat(AT_FDCWD, "foo", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0664, st_size=6, ...}) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
splice(3, NULL, 1, NULL, 1048576, 0)    = -1 EINVAL (Invalid argument)
pipe([4, 5])                            = 0
fcntl(4, F_SETPIPE_SZ, 1048576)         = 1048576
splice(3, NULL, 5, NULL, 1048576, 0)    = 6
splice(4, NULL, 1, NULL, 6, 0hello
```

```
)          = 6
splice(3, NULL, 5, NULL, 1048576, 0)    = 0
close(5)                                = 0
close(4)                                = 0
close(3)                                = 0
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NULL) = 0
munmap(0x708dbfb73000, 12288)           = 0
exit_group(0)                           = ?
+++ exited with 0 +++
pai@pai-H610M-H-DDR4:~/KMUTT/cpe333_lab01_operating_system/Lab12$ ^C
pai@pai-H610M-H-DDR4:~/KMUTT/cpe333_lab01_operating_system/Lab12$ ^C
pai@pai-H610M-H-DDR4:~/KMUTT/cpe333_lab01_operating_system/Lab12$ echo hello > foo
pai@pai-H610M-H-DDR4:~/KMUTT/cpe333_lab01_operating_system/Lab12$ strace cat foo
execve("/usr/bin/cat", ["cat", "foo"], 0x7ffef7209088 /* 82 vars */) = 0
brk(NULL)                               = 0x5f995782a000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7d64f144a000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=72063, ...}) = 0
mmap(NULL, 72063, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7d64f1438000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7d64f1405000
mmap(0x7d64f140c000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7000
mmap(0x7d64f142d000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7d64
mmap(0x7d64f1434000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2f000
mmap(0x7d64f1436000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=187120, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7d64f13d7000
mmap(0x7d64f13db000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000
mmap(0x7d64f13ff000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7d64
mmap(0x7d64f1403000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2c000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=1198376, ...}) = 0
mmap(NULL, 1200152, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7d64f12b1000
mmap(0x7d64f12c3000, 655360, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1200
```

```
mmap(0x7d64f1363000, 466944, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xb2000) = 0x7d
mmap(0x7d64f13d5000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x12300
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2190608, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7d64f1000000
mmap(0x7d64f1028000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x280
mmap(0x7d64f11c0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c0000) = 0x7c
mmap(0x7d64f120e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20e0
mmap(0x7d64f1214000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=699056, ...}) = 0
mmap(NULL, 701112, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7d64f0f54000
mmap(0x7d64f0f57000, 507904, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3000
mmap(0x7d64f0fd3000, 176128, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7f000) = 0x7d6
mmap(0x7d64f0ffe000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xa9000
close(3)                                = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7d64f12af000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7d64f12ac000
arch_prctl(ARCH_SET_FS, 0x7d64f12acb00) = 0
set_tid_address(0x7d64f12ad128)         = 27228
set_robust_list(0x7d64f12acde0, 24)     = 0
rseq(0x7d64f12ac680, 0x21, 0, 0x53053053) = 0
mprotect(0x7d64f120e000, 16384, PROT_READ) = 0
mprotect(0x7d64f0ffe000, 4096, PROT_READ) = 0
mprotect(0x7d64f13d5000, 4096, PROT_READ) = 0
mprotect(0x7d64f1403000, 4096, PROT_READ) = 0
mprotect(0x7d64f1434000, 4096, PROT_READ) = 0
mprotect(0x5f993c85d000, 1462272, PROT_READ) = 0
mprotect(0x7d64f148f000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\xb4\x52\xd5\xc1\xc6\xbb\x06\xb8", 8, GRND_NONBLOCK) = 8
munmap(0x7d64f1438000, 72063)           = 0
statfs("/sys/fs/selinux", 0x7ffcfc2de1b0) = -1 ENOENT (No such file or directory)
statfs("/selinux", 0x7ffcfc2de1b0)      = -1 ENOENT (No such file or directory)
brk(NULL)                               = 0x5f995782a000
brk(0x5f995784b000)                     = 0x5f995784b000
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
```

```
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tpr"..., 1024) = 369
read(3, "", 1024)                       = 0
close(3)                                = 0
access("/etc/selinux/config", F_OK)     = -1 ENOENT (No such file or directory)
fcntl(0, F_GETFD)                       = 0
fcntl(1, F_GETFD)                       = 0
fcntl(2, F_GETFD)                       = 0
rt_sigaction(SIGPIPE, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
poll([{fd=0, events=0}, {fd=1, events=0}, {fd=2, events=0}], 3, 0) = 0 (Timeout)
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_RESTART, sa_r
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "5f993bef4000-5f993bfe6000 r--p 0"..., 1024) = 1024
read(3, "0ffe000-7d64f0fff000 r--p 000a90"..., 1024) = 1024
read(3, "00 08:12 6293003                "..., 1024) = 1024
read(3, "             /usr/lib/x86_64-lin"..., 1024) = 1024
read(3, "2 6292997                     /us"..., 1024) = 650
lseek(3, 0, SEEK_CUR)                   = 4746
lseek(3, 4662, SEEK_SET)                = 4662
close(3)                                = 0
sched_getaffinity(27228, 32, [0 1 2 3 4 5 6 7 8 9 10 11]) = 8
rt_sigaction(SIGSEGV, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
sigaltstack(NULL, {ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=0}) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7d64f144
mprotect(0x7d64f1447000, 4096, PROT_NONE) = 0
sigaltstack({ss_sp=0x7d64f1448000, ss_flags=0, ss_size=8192}, NULL) = 0
gettid()                                = 27228
rt_sigaction(SIGSEGV, {sa_handler=0x5f993c2190c0, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA
rt_sigaction(SIGBUS, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGBUS, {sa_handler=0x5f993c2190c0, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_
prctl(PR_GET_AUXV, 0x7ffcfc2ddca0, 0x200, 0, 0) = 448
statx(AT_FDCWD, "/usr/share/coreutils/locales/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=
statx(AT_FDCWD, "/usr/share/coreutils/locales/uucore", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_ma
openat(AT_FDCWD, "/usr/share/coreutils/locales/uucore/en-US.ftl", O_RDONLY|O_CLOEXEC) = 3
statx(3, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, s
read(3, "# Common strings shared across a"..., 4080) = 4080
read(3, "", 32)                         = 0
close(3)                                = 0
statx(AT_FDCWD, "/usr/share/coreutils/locales/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=
openat(AT_FDCWD, "/usr/share/coreutils/locales/cat/en-US.ftl", O_RDONLY|O_CLOEXEC) = -1 ENOENT (
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_RESTART, sa_r
```

```
rt_sigaction(SIGSEGV, {sa_handler=SIG_DFL, sa_mask=~[RTMIN RT_1], sa_flags=SA_RESTORER, sa_resto
rt_sigaction(SIGBUS, {sa_handler=SIG_DFL, sa_mask=~[RTMIN RT_1], sa_flags=SA_RESTORER, sa_resto
ioctl(1, TCGETS2, {c_iflag=BRKINT|ICRNL|IXON|IXANY|IMAXBEL|IUTF8, c_oflag=NL0|CR0|TAB0|BS0|VT0|
brk(0x5f995786c000)                     = 0x5f995786c000
statx(AT_FDCWD, "foo", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, stx_
openat(AT_FDCWD, "foo", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0664, st_size=6, ...}) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
splice(3, NULL, 1, NULL, 1048576, 0)    = -1 EINVAL (Invalid argument)
pipe([4, 5])                            = 0
fcntl(4, F_SETPIPE_SZ, 1048576)         = 1048576
splice(3, NULL, 5, NULL, 1048576, 0)    = 6
splice(4, NULL, 1, NULL, 6, 0hello
)          = 6
splice(3, NULL, 5, NULL, 1048576, 0)    = 0
close(5)                                = 0
close(4)                                = 0
close(3)                                = 0
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NULL) = 0
munmap(0x7d64f1447000, 12288)           = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```